Software Requirements Specification (SRS)

EcoTrack — Sustainable GenAI Impact Assessment Tool

Version: *1.1 (Refactored — Firebase + Next.js Stack)* ***Date:*** *March 2026*

---

1. Introduction

1.1 Purpose

This document defines the software requirements for EcoTrack. It reflects the actual technology decisions made by the team: Next.js (frontend + backend API routes), Firebase (authentication, Firestore database, cloud functions), and JavaScript throughout. The scope is scoped to organisation-level registration and login only — no team member or multi-user management in this version.

1.2 Product Overview

EcoTrack is a web platform that acts as a proxy between an organisation's application and AI API providers (OpenAI, Anthropic, Google). It measures the energy consumption and carbon emissions of every AI API call the organisation makes, displays the data on a dashboard, generates monthly audit reports, and recommends ways to reduce environmental impact.

1.3 Scope

This version covers:

- *Organisation registration and login (Firebase Auth)*
- *Onboarding wizard (provider key setup, EcoTrack key generation, proxy integration guide)*
- *API key management (EcoTrack keys and provider keys)*
- *Proxy-based API forwarding via Next.js API routes*
- *Per-call energy and CO_2 calculation*
- *Dashboard with charts and sustainability score*
- *Project-level tracking*
- *Monthly audit report generation (PDF)*

- *Recommendation engine (rule-based)*
- *Model comparison leaderboard*

Out of scope for this version: team member invitations, multi-user roles within an org, billing, mobile app, streaming response support.

1.4 Definitions

- ***EcoTrack key** — API key generated by EcoTrack and given to the org. Used to authenticate with the proxy.*
- ***Provider key** — The org's own API key for OpenAI/Anthropic/Google. Stored encrypted in Firestore.*
- ***Proxy** — Next.js API route that intercepts and forwards AI API calls.*
- ***Org** — Organisation. The primary account entity. One org = one account in this version.*
- ***Wh** — Watt-hour. Unit of energy per API call.*
- ***kWh** — Kilowatt-hour. Aggregated energy unit shown on dashboard.*
- ***gCO_2/Wh** — Grams of CO_2 per watt-hour. Carbon intensity of a power grid region.*
- ***Firestore** — Firebase's NoSQL cloud database.*
- ***Firebase Auth** — Firebase's authentication service.*
- ***Next.js API route** — Server-side JavaScript handler in Next.js under /pages/api/ or /app/api/.*

---

2. Overall Description

2.1 Technology Stack

***Frontend:** Next.js 14 (App Router), React, Tailwind CSS, Recharts (for dashboard charts)*

***Backend:** Next.js API routes (JavaScript) — handles proxy forwarding, key management, dashboard data, report generation*

***Authentication:** Firebase Authentication (email/password)*

***Database:** Firebase Firestore (NoSQL) — stores org data, keys, call logs, projects, reports, recommendations*

***File Storage:** Firebase Storage — stores generated PDF reports*

***Scheduled Jobs:** Firebase Cloud Functions with scheduled triggers — monthly report generation, daily recommendation computation*

***Email:** Firebase Extensions (Trigger Email) or a direct SendGrid API call from a Cloud Function*

PDF Generation: *pdfkit (Node.js library) or jsPDF — called from a Next.js API route or Cloud Function*

Encryption: *Node.js built-in crypto module (AES-256-GCM) for provider key encryption*

Hosting: *Vercel (Next.js) + Firebase (backend services)*

2.2 Why This Stack Works

Next.js API routes replace Spring Boot entirely. Each route is a serverless JavaScript function. The proxy endpoint is just one API route file that receives requests, validates keys, forwards to OpenAI, logs to Firestore, and returns the response. Firebase handles auth tokens, so you do not need to build JWT logic manually. Firestore handles the database with no schema setup required — you just write JSON documents.

2.3 User Class

In this version there is exactly one user class: the ***Organisation Admin****. This is the person who registers the org, sets up keys, views the dashboard, and downloads reports. There are no separate member roles, auditor roles, or system admin roles in v1.*

2.4 Operating Environment

- *Node.js 18+ (required by Next.js 14 and Firebase Admin SDK)*
- *Deployed on Vercel (Next.js app) and Firebase (Firestore, Auth, Storage, Functions)*
- *All communication over HTTPS*
- *Browser support: Chrome, Firefox, Edge (latest 2 versions)*

2.5 Assumptions

- *The org's app can change its AI provider base URL with a one-line config change*
- *Energy-per-token values are approximations based on published research — exact values are not disclosed by providers*
- *Carbon intensity values per region are static and updated manually each quarter*
- *The hackathon demo uses real or test API keys*

---

3. Firestore Data Model

This replaces the SQL schema section. Firestore is document-based. Here is the complete collection structure.

Collection: `orgs`

One document per registered organisation.

orgs/{orgId}

```
name:          string      // "Acme Corp"
email:         string      // login email, same as Firebase Auth email
firebaseUid:   string      // Firebase Auth UID, used to link auth to org
region:        string      // "india" | "us" | "eu" | "norway"
industry:      string      // "fintech" | "healthtech" | "ecommerce" | ...
size:          string      // "1-10" | "11-50" | "51-200" | "200+"
aiProviders:   array       // ["openai", "anthropic", "google"]
plan:          string      // "free" | "pro"
createdAt:     timestamp
updatedAt:     timestamp
```

Collection: ecoKeys

One document per EcoTrack API key issued to the org.

ecoKeys/{keyId}

```
orgId:         string      // reference to orgs/{orgId}
projectId:     string|null  // null means global key
keyHash:       string      // bcrypt hash of the full key — never store plaintext
keyPrefix:     string      // first 12 chars, shown in UI e.g. "eco-sk-org12"
name:          string      // user-given label e.g. "Production"
revoked:       boolean
createdAt:     timestamp
lastUsedAt:    timestamp|null
```

Collection: `providerKeys`

One document per AI provider key the org connects.

providerKeys/{keyId}

```
orgId:          string
provider:       string      // "openai" | "anthropic" | "google"
encryptedKey:   string      // AES-256-GCM encrypted key
iv:             string      // AES initialisation vector (hex)
authTag:        string      // AES GCM auth tag (hex)
nickname:       string      // "Production OpenAI key"
createdAt:      timestamp
```

Collection: `projects`

projects/{projectId}

```
orgId:          string
name:           string
description:    string
environment:    string      // "production" | "staging" | "development"
archived:       boolean
createdAt:      timestamp
```

Collection: `callLogs`

One document per API call proxied. This is the most important collection — everything on the dashboard comes from here.

callLogs/{logId}

```
orgId:          string
projectId:      string|null
provider:       string      // "openai" | "anthropic" | "google"
model:          string      // "gpt-4" | "gpt-3.5-turbo" | ...
promptTokens:   number
completionTokens: number
totalTokens:    number
energyWh:       number      // calculated
co2Grams:       number      // calculated
region:         string      // copied from org at time of call
status:         string      // "success" | "failed"
createdAt:      timestamp
```

Collection: `reports`

reports/{reportId}

```
orgId:          string
projectId:      string|null
periodStart:    timestamp
periodEnd:      timestamp
format:         string      // "pdf" | "csv"
storageUrl:     string      // Firebase Storage download URL
generatedAt:    timestamp
```

Collection: `recommendations`

recommendations/{recId}

```
orgId:          string
type:           string      // "model_switch" | "context_length" | "staging"
title:          string
description:    string
energySavingWh: number
co2SavingGrams: number
dismissed:      boolean
dismissedAt:    timestamp|null
createdAt:      timestamp
```

Subcollection: `modelRegistry` (top-level collection)

modelRegistry/{modelId}

```
modelName:      string      // "gpt-4"
provider:       string      // "openai"
whPer1kTokens:  number      // 5.0
taskTags:       array       // ["complex reasoning", "code generation"]
updatedAt:      timestamp
```

Collection: `regionCarbonIntensity` (top-level collection)

regionCarbonIntensity/{regionId}

```
regionCode:     string      // "india"
regionName:     string      // "India"
```

gco2PerWh: number // 0.82

updatedAt: timestamp

---

4. Authentication Flow (Firebase Auth)

4.1 Registration

The org admin visits /signup. They fill the registration form. On submit, the frontend calls Firebase Auth `createUserWithEmailAndPassword(email, password)`*. Firebase creates the auth user and returns a UID and ID token.*

Immediately after, the frontend calls your Next.js API route POST /api/auth/register, sending the ID token plus the org form data (name, region, industry, size, aiProviders). The API route verifies the ID token using Firebase Admin SDK, then creates the org document in Firestore with the firebaseUid field set to the Firebase Auth UID.

From this point, the org document and the Firebase Auth user are permanently linked via firebaseUid.

4.2 Login

The org admin visits /login. They enter email and password. Frontend calls Firebase Auth `signInWithEmailAndPassword(email, password)`*. Firebase returns an ID token on success.*

The frontend stores this token (Firebase handles this automatically in the browser via persistence). For all subsequent API route calls, the frontend sends the ID token in the Authorization header. Every protected API route verifies this token using Firebase Admin SDK before doing anything.

4.3 Logout

Frontend calls Firebase Auth `signOut()`*. Firebase clears the local token. User is redirected to /login.*

4.4 Session Persistence

Firebase Auth handles session persistence automatically in the browser. The ID token expires after 1 hour but Firebase auto-refreshes it using the refresh token. No manual refresh logic needed.

4.5 Protected Routes (Next.js)

All pages except /, /login, and /signup are protected. Use Next.js middleware (middleware.js at project root) to check for a valid Firebase session cookie on every request. If not found, redirect to /login.

---

5. Functional Requirements

5.1 Registration and Login

FR-01 Registration Form *The /signup page shall collect: full name, work email, password, organisation name, organisation size (dropdown), industry (dropdown), country/region (dropdown mapped to carbon intensity region), and intended AI providers (checkboxes). All fields except industry are required.*

FR-02 Firebase Auth Account Creation *On form submit, the system shall call Firebase Auth createUserWithEmailAndPassword. On success, it shall call POST /api/auth/register to create the Firestore org document. On failure (email already exists, weak password), the system shall display Firebase's error message in plain English.*

FR-03 Org Document Creation *POST /api/auth/register shall verify the Firebase ID token, extract the UID, and write the org document to Firestore. It shall return the orgId to the frontend, which stores it in localStorage for subsequent requests.*

FR-04 Login *The /login page shall call Firebase Auth signInWithEmailAndPassword. On success, Firebase stores the session. The frontend redirects to /dashboard.*

FR-05 Login Error Handling *On wrong password or unregistered email, the system shall display a user-friendly message. The raw Firebase error code shall not be shown to the user.*

FR-06 Logout *Clicking logout shall call Firebase Auth signOut and redirect to /login. All locally stored org data shall be cleared from localStorage.*

FR-07 Route Protection *All routes under /dashboard, /keys, /projects, /reports, /recommendations, /settings, and /onboarding shall require a valid Firebase session. If no session exists, the user shall be redirected to /login.*

5.2 Onboarding

FR-08 Redirect After Registration *On successful registration, the system shall redirect to /onboarding instead of /dashboard. The onboarding flag in the org document (onboardingComplete: false) controls this. Once onboarding is marked complete, the user goes directly to /dashboard on login.*

FR-09 Onboarding Step 1 — Add Provider Key *The user selects a provider (OpenAI / Anthropic / Google), enters their API key, and gives it a nickname. On clicking "Test and Save", the frontend calls POST /api/keys/provider/test which makes a minimal real request to the provider (e.g. GET models list for OpenAI) using the submitted key. If the test passes, the key is encrypted using AES-256-GCM in the API route and saved to the providerKeys collection. If the test fails, the provider's error is shown.*

FR-10 Onboarding Step 2 — Receive EcoTrack Key *The system shall generate a unique EcoTrack API key in the format eco-sk-{orgId first 8 chars}-{32 random hex chars}. The full key is shown once on screen with a copy button. The system stores only the bcrypt hash and the first 12 characters (prefix) in Firestore. The full key is never stored. This is handled by POST /api/keys/eco/generate.*

FR-11 Onboarding Step 3 — Create First Project *The user enters a project name, optional description, and selects environment (Production / Staging / Development). POST /api/projects creates the project document and returns a project-scoped EcoTrack key.*

FR-12 Onboarding Step 4 — Integration Verification *The page displays the proxy endpoint URL and code snippets. It shows a live status panel that polls GET /api/dashboard/live-feed every 5 seconds. When the first proxied call appears, the panel lights up showing the detected call. A "Complete setup" button marks onboardingComplete: true in Firestore.*

5.3 API Key Management

FR-13 List EcoTrack Keys *GET /api/keys/eco shall return all EcoTrack keys for the authenticated org. Each key object contains: id, name, prefix (first 12 chars), projectId, revoked, createdAt, lastUsedAt. The full key hash is never returned.*

FR-14 Create EcoTrack Key *POST /api/keys/eco shall generate a new EcoTrack key, store the hash in Firestore, and return the full key once in the response. The frontend shows it with a copy button and a warning that it will not be shown again.*

FR-15 Revoke EcoTrack Key *DELETE /api/keys/eco/{keyId} shall set revoked: true in Firestore. The proxy checks this field on every request. Revoked keys are rejected immediately with HTTP 401.*

FR-16 List Provider Keys *GET /api/keys/provider shall return all provider key documents for the org. The encryptedKey and iv fields shall never be included in the response. Only id, provider, nickname, and createdAt are returned.*

FR-17 Add Provider Key *POST /api/keys/provider shall accept provider and apiKey (plaintext) plus nickname. The API route encrypts the key using AES-256-GCM (using ENCRYPTION_KEY from environment variables as the secret), stores encryptedKey, iv, and authTag in Firestore, and returns success. The plaintext key is never stored or logged.*

FR-18 Remove Provider Key *DELETE /api/keys/provider/{keyId} shall delete the provider key document. If any active EcoTrack key depends on this provider, the response shall include a warning.*

5.4 Proxy and API Forwarding

This is the core technical feature. It is implemented as a Next.js API route.

FR-19 Proxy Endpoint — OpenAI *The system shall expose POST /api/proxy/openai/v1/chat/completions. This route accepts requests in the exact same format as OpenAI's chat completions API.*

FR-20 Proxy Endpoint — Anthropic *The system shall expose POST /api/proxy/anthropic/v1/messages.*

FR-21 Proxy Endpoint — Google *The system shall expose POST /api/proxy/google/v1/generateContent.*

FR-22 EcoTrack Key Validation *Every proxy request must include Authorization: Bearer eco-sk-... in the headers. The API route shall:*

1. *Extract the key from the header*
2. *Query Firestore ecoKeys collection where keyPrefix matches the first 12 chars of the submitted key*
3. *bcrypt-compare the submitted key against the stored hash*
4. *Check revoked is false*
5. *If any check fails, return HTTP 401 immediately*

FR-23 Org and Provider Key Retrieval *After validating the EcoTrack key, the route retrieves the org document (for region) and the matching provider key document (for encryptedKey, iv, authTag). It decrypts the provider key in memory using the ENCRYPTION_KEY environment variable.*

FR-24 Request Forwarding *The route constructs an outgoing HTTP request to the AI provider using the decrypted provider key in the Authorization header. The request body is passed through unchanged. This is done using the native Node.js fetch (available in Node 18+).*

FR-25 Response Pass-Through *The provider's response body and status code are returned to the caller unchanged. The caller cannot distinguish a proxied response from a direct provider response.*

FR-26 Async Metrics Logging *After returning the response, the route triggers an async Firestore write (does not await it, so it does not delay the response) containing: orgId, projectId, model, token counts extracted from response body, energyWh, co2Grams, status, createdAt.*

FR-27 Energy Calculation Logic

```
// In the proxy API route, after receiving OpenAI response:


const usage = responseBody.usage

const totalTokens = usage.total_tokens

const model = requestBody.model


// Look up from modelRegistry collection in Firestore

const modelDoc = await db.collection('modelRegistry')

                    .where('modelName', '==', model)

                    .get()

const whPer1k = modelDoc.docs[0].data().whPer1kTokens


// Look up carbon intensity for org's region

const regionDoc = await db.collection('regionCarbonIntensity')

                     .where('regionCode', '==', org.region)

                     .get()

const gco2PerWh = regionDoc.docs[0].data().gco2PerWh
```

```
const energyWh = (totalTokens / 1000) * whPer1k
const co2Grams = energyWh * gco2PerWh

// Write to callLogs
await db.collection('callLogs').add({
  orgId, projectId, model, provider,
  promptTokens: usage.prompt_tokens,
  completionTokens: usage.completion_tokens,
  totalTokens,
  energyWh,
  co2Grams,
  region: org.region,
  status: 'success',
  createdAt: admin.firestore.FieldValue.serverTimestamp()
})
```

FR-28 Proxy Encryption Implementation

```
// utils/encryption.js

const crypto = require('crypto')
const ALGORITHM = 'aes-256-gcm'
const KEY = Buffer.from(process.env.ENCRYPTION_KEY, 'hex') // 32-byte hex key
```

```
function encrypt(plaintext) {
  const iv = crypto.randomBytes(12)
  const cipher = crypto.createCipheriv(ALGORITHM, KEY, iv)
  const encrypted = Buffer.concat([
    cipher.update(plaintext, 'utf8'),
    cipher.final()
  ])
  const authTag = cipher.getAuthTag()
  return {
    encryptedKey: encrypted.toString('hex'),
    iv: iv.toString('hex'),
    authTag: authTag.toString('hex')
  }
}

function decrypt(encryptedKey, iv, authTag) {
  const decipher = crypto.createDecipheriv(
    ALGORITHM, KEY, Buffer.from(iv, 'hex')
  )
  decipher.setAuthTag(Buffer.from(authTag, 'hex'))
  const decrypted = Buffer.concat([
    decipher.update(Buffer.from(encryptedKey, 'hex')),
    decipher.final()
  ])
```

```
  return decrypted.toString('utf8')
}


module.exports = { encrypt, decrypt }
```

FR-29 Latency *The proxy route shall add no more than 200ms of overhead. The Firestore key validation query is the main cost. If this becomes a bottleneck, key validation results shall be cached in memory (Node.js Map with 5-minute TTL) within the serverless function instance.*

FR-30 Error Forwarding *If the AI provider returns a 4xx or 5xx, the proxy shall forward that response unchanged and log the call with status: 'failed'.*

5.5 Dashboard

FR-31 Summary Cards *GET /api/dashboard/summary?from=&to=&projectId= shall query callLogs for the org, filtered by date range and optionally by projectId, and return: totalCalls, totalEnergyKwh, totalCo2Kg, sustainabilityScore.*

FR-32 Sustainability Score Calculation

```
// In /api/dashboard/summary


const avgWhPer1kTokens = (totalEnergyWh / totalTokens) * 1000

const maxWhPer1kTokens = 5.0  // GPT-4, the least efficient model

const score = Math.max(0, Math.round(

  100 - (avgWhPer1kTokens / maxWhPer1kTokens) * 80

))
```

FR-33 Energy Over Time *GET /api/dashboard/energy-over-time?from=&to= shall return an array of {date, energyWh} objects grouped by day, queried from callLogs.*

FR-34 CO_2 by Model *GET /api/dashboard/co2-by-model shall return an array of {model, totalCo2Grams} objects for the current month.*

FR-35 Calls by Project *GET /api/dashboard/calls-by-project shall return an array of {projectName, callCount} for the current month.*

FR-36 Live Feed *GET /api/dashboard/live-feed shall return the last 20 callLogs documents ordered by createdAt descending. The frontend polls this every 10 seconds.*

FR-37 Filters *All dashboard API routes shall accept from and to query parameters (ISO date strings) and an optional projectId parameter.*

5.6 Projects

FR-38 Create Project *POST /api/projects creates a new project document in Firestore and generates a project-scoped EcoTrack key automatically.*

FR-39 List Projects *GET /api/projects returns all non-archived projects for the org with summary stats (call count, energy, CO_2) for the current month pulled from callLogs.*

FR-40 Project Detail *GET /api/projects/{projectId}/summary returns the same dashboard metrics but filtered to that project only.*

FR-41 Archive Project *PATCH /api/projects/{projectId} with body {archived: true} sets the archived field. The corresponding EcoTrack key is also revoked.*

5.7 Reports

FR-42 Generate Report *POST /api/reports/generate accepts periodStart, periodEnd, projectId (optional), and format (pdf or csv). It queries callLogs, computes all metrics, generates the file, uploads to Firebase Storage, saves a report document in Firestore, and returns the download URL.*

FR-43 PDF Contents *The PDF shall include: org name, report period, generation date, total calls, total energy (kWh), total CO_2 (kg), sustainability score, energy by model table, energy by project table, day-by-day energy data, comparison to previous period, and top 5 most energy-intensive calls.*

FR-44 CSV Contents *One row per call log entry with columns: timestamp, project, provider, model, promptTokens, completionTokens, totalTokens, energyWh, co2Grams, region, status.*

FR-45 Scheduled Monthly Report *A Firebase Cloud Function scheduled for the 1st of each month shall auto-generate a PDF report for every org covering the previous month, upload it to Firebase Storage, save the report document, and email the download link to the org's registered email.*

FR-46 Report History *GET /api/reports returns all report documents for the org ordered by generatedAt descending.*

5.8 Recommendations

FR-47 Model Switch Recommendation *A daily Firebase Cloud Function queries each org's callLogs for the past 7 days. If more than 30% of calls used GPT-4 or Claude Opus for tasks with fewer than 500 tokens (indicating simple tasks), a recommendation document is created suggesting GPT-3.5 or Claude Sonnet with estimated savings calculated.*

FR-48 Context Length Recommendation *If more than 20% of an org's calls in the past 7 days had promptTokens over 2000, a recommendation is created suggesting context truncation.*

FR-49 Staging Waste Recommendation *If a staging-environment project made more than 200 calls in the past day, a recommendation is created suggesting API mocking in staging.*

FR-50 List Recommendations *GET /api/recommendations returns all non-dismissed recommendations for the org ordered by energySavingWh descending.*

FR-51 Dismiss Recommendation *PATCH /api/recommendations/{recId} with {dismissed: true} sets dismissed: true and dismissedAt to now in Firestore.*

5.9 Model Comparison

FR-52 Leaderboard *GET /api/models/leaderboard returns all documents from the modelRegistry collection sorted by whPer1kTokens ascending (most efficient first), with a computed relative efficiency percentage added.*

FR-53 What-If Calculator *POST /api/models/whatif accepts currentModel and monthlyTokens (number). It returns an array of alternative models with estimated energySavingWh and co2SavingGrams if the org switched to each alternative.*

---

6. Next.js File Structure

```
/app
  /page.js                    ← landing page
  /login/page.js
  /signup/page.js
  /onboarding/page.js
  /dashboard/page.js
  /keys/page.js
  /projects/page.js
  /projects/[id]/page.js
  /reports/page.js
  /recommendations/page.js
  /models/page.js

/app/api
  /auth/register/route.js     ← creates org in Firestore after Firebase Auth signup
  /keys/eco/route.js          ← GET list, POST create
  /keys/eco/[id]/route.js     ← DELETE revoke
  /keys/provider/route.js     ← GET list, POST add
  /keys/provider/[id]/route.js← DELETE remove
  /keys/provider/test/route.js← POST test a provider key
  /proxy/openai/[...path]/route.js   ← catches all OpenAI proxy routes
  /proxy/anthropic/[...path]/route.js
  /proxy/google/[...path]/route.js
  /dashboard/summary/route.js
```

```
  /dashboard/energy-over-time/route.js
  /dashboard/co2-by-model/route.js
  /dashboard/calls-by-project/route.js
  /dashboard/live-feed/route.js
  /projects/route.js
  /projects/[id]/route.js
  /projects/[id]/summary/route.js
  /reports/route.js
  /reports/generate/route.js
  /recommendations/route.js
  /recommendations/[id]/route.js
  /models/leaderboard/route.js
  /models/whatif/route.js

/lib
  /firebase.js          ← Firebase client SDK init
  /firebaseAdmin.js     ← Firebase Admin SDK init (for API routes)
  /encryption.js        ← AES-256-GCM encrypt/decrypt
  /energyCalc.js        ← energy and CO₂ calculation functions
  /ecoKeyGen.js         ← EcoTrack key generation and hashing

/middleware.js          ← route protection (checks Firebase session)

/functions              ← Firebase Cloud Functions
```

```
/monthlyReport.js          ← scheduled monthly PDF generation
/dailyRecommendations.js   ← daily recommendation engine
```

---

7. Environment Variables

```
# Firebase Admin SDK
FIREBASE_PROJECT_ID=
FIREBASE_CLIENT_EMAIL=
FIREBASE_PRIVATE_KEY=

# Firebase Client SDK (public, prefixed with NEXT_PUBLIC_)
NEXT_PUBLIC_FIREBASE_API_KEY=
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=
NEXT_PUBLIC_FIREBASE_PROJECT_ID=
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=
NEXT_PUBLIC_FIREBASE_APP_ID=

# Encryption
ENCRYPTION_KEY=          # 64-char hex string (32 bytes for AES-256)

# Email
SENDGRID_API_KEY=
```

```
# App

NEXT_PUBLIC_APP_URL=     # e.g. https://ecotrack.vercel.app
```

---

8. Non-Functional Requirements

NFR-01 Latency *— The proxy route shall return the AI provider's response within 200ms of receiving it (excluding provider processing time).*

NFR-02 Security *— Provider keys stored AES-256-GCM encrypted. EcoTrack keys stored as bcrypt hashes. Full keys never logged or returned after creation. All routes over HTTPS.*

NFR-03 Firestore Security Rules *— Firestore rules shall deny all direct client access to providerKeys, ecoKeys (hash field), and callLogs. These collections are only accessible via Firebase Admin SDK in API routes.*

NFR-04 Scalability *— Next.js API routes on Vercel are serverless and scale automatically. Firestore scales automatically. No manual scaling configuration required for the hackathon scope.*

NFR-05 Usability *— A developer shall be able to complete proxy integration within 15 minutes using the onboarding guide.*

NFR-06 Data Retention *— Call logs are retained indefinitely in Firestore for the hackathon. A soft-delete or TTL policy can be added in a future version.*

---

9. Firestore Security Rules

```
rules_version = '2';

service cloud.firestore {

  match /databases/{database}/documents {



    // Orgs — readable and writable only by the matching Firebase Auth user

    match /orgs/{orgId} {
```

```
  allow read, write: if request.auth != null
    && request.auth.uid == resource.data.firebaseUid;
}

// All other collections — no direct client access
// Only Firebase Admin SDK (used in API routes) can access these
match /ecoKeys/{keyId} {
  allow read, write: if false;
}
match /providerKeys/{keyId} {
  allow read, write: if false;
}
match /callLogs/{logId} {
  allow read, write: if false;
}
match /projects/{projectId} {
  allow read, write: if false;
}
match /reports/{reportId} {
  allow read, write: if false;
}
match /recommendations/{recId} {
  allow read, write: if false;
}
```

```
    match /modelRegistry/{modelId} {
      allow read: if true;   // public read — anyone can see model energy data
      allow write: if false; // only admin SDK can write
    }
    match /regionCarbonIntensity/{regionId} {
      allow read: if true;
      allow write: if false;
    }
  }
}
```

---

10. Acceptance Criteria (Hackathon Demo)

1. *A new org can register with email and password, complete the 4-step onboarding, and receive a working EcoTrack API key within 10 minutes.*
2. *A test API call made through the proxy appears in the live feed on the dashboard within 10 seconds, showing correct model, tokens, energy (Wh), and CO_2 (g).*
3. *The dashboard shows energy over time, CO_2 by model, calls by project, and sustainability score correctly after at least 10 proxied calls.*
4. *A PDF report can be generated and downloaded for the demo session period.*
5. *At least two recommendations appear after the demo call pattern is established.*
6. *Revoking an EcoTrack key causes the next proxied call using that key to return HTTP 401 immediately.*
7. *The model leaderboard correctly ranks all supported models by energy efficiency.*
8. *Logging out and logging back in restores the full dashboard state from Firestore.*

---

11. Out of Scope (This Version)

- *Team member invitations and multi-user org management*
- *Role-based access (Admin vs Member)*

- *Billing and payment*
- *Streaming API response support*
- *Image, audio, or embedding model tracking*
- *Mobile application*
- *Self-hosted AI model support*
- *SOC 2 or regulatory compliance certification*
- *Multi-region Firestore deployment*